



**INSTITUTO DE EDUCACIÓN SECUNDARIA LA MOLA
NOVELDA**

DESARROLLO DE APLICACIONES WEB

Curso Académico 2025/2026

Desarrollo Web en Entorno Cliente

U02_A03_Ejercicio

Autor: Segura Abad, Mario

Fecha: 12 de Octubre, 2025

INTRODUCCIÓN

En el ejercicio “Clases y herencia prototípica (delegación)” deberemos crear una clase denominada ‘Animal’ con sus propiedades correspondientes (‘nombre’ y ‘edad’) y un método llamado ‘correr()’ que imprima el consola dicha acción. Asimismo, crearemos una clase hija denominada ‘Perro’ que herede de ‘Animal’ con una propiedad adicional y un método adicional de esa clase.

Crearemos una instancia de la clase ‘Perro’ y estableceremos ‘nombre’, ‘raza’ y ‘edad’, y llamaremos a los métodos correspondientes ‘comer()’ y ‘ladrar()’ para comprobar su correcto funcionamiento.

ENUNCIADO

Clases y herencia prototípica (delegación)

Crema una clase llamada **Animal** con las propiedades **nombre** y **edad** y un método llamado **comer()** que imprima en consola “El animal está comiendo”. Luego crea una clase hija llamada **Perro** que herede de la clase **Animal** y añada una propiedad adicional **raza** y un método llamado **ladrar()** que imprima en consola “¡Guau!”.

Crema una instancia de la clase **Perro**, establecer un **nombre**, **edad** y **raza**, y llamar a los métodos **comer()** y **ladrar()** para comprobar que funcionan correctamente y consulta sus propiedades.

Ejemplo de implementación:



```
> // Autor: Mario Segura Abad
// Fecha: 12/10/2025

// Clase base Animal
function Animal(nombre, edad) {
  this.nombre = nombre;
  this.edad = edad;
}

Animal.prototype.comer = function () {
  console.log("El animal está comiendo");
};

// Clase hija Perro que hereda de Animal
function Perro(nombre, edad, raza) {
  Animal.call(this, nombre, edad); // Hereda propiedades
  this.raza = raza;
}

// Herencia prototípica
Perro.prototype = Object.create(Animal.prototype);
Perro.prototype.constructor = Perro;

// Método específico de Perro
Perro.prototype.ladRAR = function () {
  console.log("¡Guau!");
};

// Crear una instancia de Perro
const miPerro = new Perro("Firulais", 3, "Labrador");

// Usar los métodos
miPerro.comer(); // Hereda de Animal
miPerro.ladRAR(); // Propio de Perro
```

DESARROLLO DE LA PRÁCTICA

Práctica DWEK – Unidad 2

Descripción

Este proyecto forma parte del estudio de la Programación Orientada a Objetos en JavaScript, concretamente el uso de **clases** y la **herencia prototípica**.

El objetivo principal ha sido crear una clase base llamada Animal con propiedades y métodos comunes, y una clase hija llamada Perro que herede de Animal y añada propiedades y comportamientos específicos.

Este ejercicio permite comprender cómo se estructura la herencia en JavaScript moderno utilizando la sintaxis class, y cómo se puede extender funcionalidad de forma ordenada y reutilizable.

Contenido del archivo

Código del caso práctico extendido

En este ejercicio se ha implementado una estructura de clases en JavaScript que permite modelar animales y, en concreto, perros como una subcategoría. Se ha utilizado la sintaxis moderna de class para definir una clase base llamada Animal y una clase hija llamada Perro, que hereda de la anterior.

El objetivo es demostrar cómo se puede reutilizar código mediante herencia, cómo se inicializan propiedades con super() y cómo se añaden métodos específicos en clases derivadas.

Ejemplo de ejecución:

```
// Consultar propiedades
console.log("Nombre:", miPerro.nombre);
console.log("Edad:", miPerro.edad);
console.log("Raza:", miPerro.raza);

// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 12/10/2025");

El animal está comiendo

¡Guau!

Nombre: Firulais
Edad: 3
Raza: Labrador
Autor: Mario Segura Abad
Fecha de realización: 12/10/2025
← undefined
```

Cómo visualizarlo

Requisitos:

- Editor de código: en mi caso, Visual Studio Code.
- Navegador web para scripts con `document.write()` o `prompt()`.
- Node.js o la consola del navegador (F12) para ejecutar scripts con `console.log()`.

Pasos

1. Abrir el archivo `.js` en el editor de código.
2. Ejecutar el script en el navegador o consola del navegador, según el tipo.
3. Observar el resultado en pantalla o consola.
4. Revisar las capturas de ejecución incluidas en la carpeta correspondiente.

Objetivos de la práctica

- Comprender el uso de clases en JavaScript.
- Aplicar herencia prototípica mediante la palabra clave `extends`.
- Utilizar `super()` para heredar propiedades del constructor padre.
- Crear instancias de clases hijas y acceder a métodos heredados y propios.
- Documentar correctamente el código con nombre, fecha y capturas.
- Presentar el proyecto en formato PDF junto con los archivos fuentes y pantallazos.

CONCLUSIÓN

Esta práctica ha permitido aplicar los conceptos de clases y herencia en JavaScript de forma clara y funcional.

Se ha comprobado cómo extender una clase base para crear objetos más específicos, reutilizando propiedades y métodos.

Además, se ha reforzado el uso de `super()` para inicializar correctamente las propiedades heredadas, y se ha trabajado la documentación técnica del proyecto.